

Changelog - PDF-cli

Todas as mudanças notáveis do projeto serão documentadas neste arquivo.

[0.8.0] - 2025-11-20 (Fase 8 - Distribuição Portátil e Scripts de Build Cross-platform)

Adicionado

■ **Scripts de Build Automatizados:** Scripts completos para gerar executáveis standalone

- `scripts/build_win.bat` - Script de build para Windows
- `scripts/build_linux.sh` - Script de build para Linux (WSL)
- Instalação automática de dependências (PyInstaller, Pillow, python3-venv)
- Separação de diretórios de build (evita conflitos entre Windows/Linux)

■ **Executáveis Standalone:** Executáveis portáteis que funcionam sem Python

- Windows: `dist/windows/pdf-cli.exe` (~37 MB)
- Linux: `dist/linux/pdf-cli` (~41 MB)
- Todas as dependências incluídas (PyMuPDF, PyPDF2, Pillow)

■ **Novos Comandos CLI:**

- `export-text` - Alias para `export-objects --types text`
- `export-images` - Extrai imagens do PDF como arquivos PNG/JPG
- `list-fonts` - Lista todas as fontes e variantes usadas no PDF

■ **Melhorias no Comando `edit-text`:**

- Flag `--all-occurrences` para editar todas as ocorrências de um texto
- Flag `--verbose` para feedback detalhado de cada modificação
- Sistema de detecção de fontes faltantes no sistema operacional
- Confirmação interativa quando há problemas de fonte

■ **Sistema de Gerenciamento de Fontes:**

- Detecção automática de fontes faltantes no sistema

- Avisos ao usuário sobre fontes necessárias para edição precisa
- Normalização de nomes de fontes para comparação consistente

■ Documentação de Build:

- `results/FASE-8-RELATORIO-BUILD-WINDOWS.md` - Relatório detalhado do build Windows
- `results/FASE-8-RELATORIO-FINAL.md` - Relatório final da Fase 8
- `scripts/README-BUILD-LINUX.md` - Guia completo de build Linux
- `dist/README.txt` - Instruções para usuários finais

Melhorado

■ CLI Help Avançado (Fase 7): Help detalhado para todos os 13 comandos

- Descrição sintética de cada comando
- Parâmetros, flags, tipos, valores padrão
- Exemplos práticos com arquivos reais
- Estrutura de JSON gerado
- Logs gerados
- Limitações conhecidas
- Comandos relacionados

■ Edição de Texto: Sistema de fallback inteligente para preservação de fontes

- Uso de PyMuPDF TextWriter para melhor preservação de fontes
- Detecção de fallback de fontes
- Notificações ao usuário sobre fontes faltantes

■ Validação de Paths: Prevenção de usar mesmo arquivo para entrada e saída

■ Sistema de Build: Separação de diretórios de build por plataforma

- Windows: `build/windows`, `dist/windows`
- Linux: `build/linux`, `dist/linux`

Corrigido

- ■ **Conflitos de Build:** Separação de diretórios evita conflitos entre builds Windows/Linux
- ■ **Imports PyInstaller:** Módulos `cli`, `app`, `core` agora são coletados corretamente

- ■ **Ambiente Virtual Linux:** Instalação automática de `python3-venv` quando necessário

Documentado

- ■■ **ApplImage no WSL:** ApplImage não pode ser gerado no WSL devido à falta de FUSE
- ■■ **Tamanho dos Executáveis:** ~37-41 MB (esperado para executáveis standalone)

Status de Implementação

- ■ **12 de 13 comandos** implementados com operações REAIS
- ■■ **1 comando** com limitação técnica documentada (`edit-table`)
- ■ **Executáveis standalone** disponíveis para Windows e Linux

[0.7.0] - 2025-11-XX (Fase 7 - HELP Avançado e Exemplos Práticos no CLI)

Adicionado

- **Help Expandido:** Help detalhado para todos os 13 comandos CLI
 - Descrições completas de funcionamento
 - Parâmetros, flags, tipos, valores padrão
 - Exemplos práticos usando arquivos reais
 - Estrutura de JSON gerado
 - Logs gerados
 - Limitações conhecidas
 - Comandos relacionados

Melhorado

- ■ **CLI Help:** Refatoração completa para `print()` puro (removido Typer/Rich)
 - ■ **Mensagens:** Todas as mensagens em português
 - ■ **Estrutura de Help:** Organização clara e informativa
-

[0.6.0] - 2025-11-XX (Fase 6 - Testes Reais e Relatório de Auditoria)

Adicionado

- ■ **Comando `export -text`**: Alias para `export-objects --types text`
- ■ **Comando `export -images`**: Extrai imagens do PDF como arquivos PNG/JPG
- ■ **Relatório de Testes**: `results/FASE-6-RELATORIO-TESTES-REAIS.md`

Melhorado

- ■ **Extração de Imagens**: Exporta imagens como arquivos separados (PNG/JPG)

[0.5.0] - 2025-11-XX (Fase 5 - Fallback Inteligente PyMuPDF + pypdf e Auditoria Completa)

Adicionado

- ■ **Comando `list -fonts`**: Lista fontes usadas no PDF
- ■ **Sistema de Gerenciamento de Fontes**: Detecção de fontes faltantes
- ■ **Flag `--include-fonts`** no `export-objects`: Inclui informações de fontes
- ■ **Notificações de Fontes**: Avisos ao usuário sobre fontes necessárias

Melhorado

- ■ **Edição de Texto**: Sistema de preservação de fontes melhorado
 - Uso de TextWriter (PyMuPDF) para melhor preservação
 - Detecção de fallback de fontes
 - Confirmação interativa quando há problemas
-

[0.4.0] - 2025-01-XX (Fase 4 - Testes, Robustez e Honestidade)

Adicionado

■ **Testes de Integração REAIS:** Suite completa de testes que executam operações reais sobre PDFs reais

- Testes para todos os comandos CLI: `export-objects`, `edit-text`, `replace-image`, `insert-object`, `edit-metadata`, `merge`, `delete-pages`, `split`, `restore-from-json`
- Testes de casos de uso comuns e edge cases
- Validação real de resultados (PDFs gerados, JSON exportado, logs)
- Cobertura >90% dos comandos principais

■ **Sistema de Logging Aprimorado (Fase 4):**

- Logs em formato JSONL para fácil processamento e auditoria
- Campos adicionais: `object_ids` (IDs de objetos alterados), `suggestions` (sugestões automáticas)
- Logs estruturados e auditáveis para conformidade pública
- Logs salvos em `./logs/operations.jsonl`

■ **Script de Validação de Honestidade:**

- `scripts/validate_honesty.py` - Valida que implementações são REAIS (sem mocks)
- Verifica uso de PyMuPDF para operações reais
- Valida estrutura de logs
- Relatório de ocorrências suspeitas

■ **Documentação Completa:**

- README atualizado com status real, limitações e cenários não atendidos
- CHANGELOG documentando todas as mudanças
- Seção "Cenários não atendidos" com detalhamento honesto

Melhorado

- ■ **Logging JSON:** Melhorado para incluir campos de auditoria (`object_ids`, `suggestions`)

- ■ **Validação de Resultados:** Testes validam resultados reais nos PDFs gerados
- ■ **Transparência:** Documentação clara sobre limitações técnicas conhecidas

Documentado

- ■■ **Limitação Técnica - `edit-table`:** Requer algoritmo de detecção de estrutura de tabelas (movido para fase final)
- ■■ **Extração Parcial:** Table, FormField, Graphic, Layer, Filter requerem algoritmos complexos de detecção

Status de Implementação

- ■ **9 de 10 comandos** implementados com operações REAIS
- ■■ **1 comando** com limitação técnica documentada (`edit-table`)

[0.3.0] - 2025-01-XX (Fase 3 - Manipulação Avançada de Objetos PDF)

Adicionado

- ■ **Comando `export-objects`:** Extrai objetos do PDF para JSON (text, image, link, annotation)
- ■ **Comando `edit-text`:** Edita objetos de texto via ID ou busca (IMPLEMENTAÇÃO REAL com PyMuPDF)
- ■ **Comando `replace-image`:** Substitui imagens mantendo posição (IMPLEMENTAÇÃO REAL)
- ■ **Comando `insert-object`:** Insere novos objetos (text, image implementados)
- ■ **Comando `restore-from-json`:** Restaura PDF via JSON (text implementado)
- ■ **Comando `edit-metadata`:** Edita metadados do PDF
- ■ **Comando `merge`:** Une múltiplos PDFs
- ■ **Comando `delete-pages`:** Exclui páginas específicas
- ■ **Comando `split`:** Divide PDF em múltiplos arquivos
- ■■ **Comando `edit-table`:** Estrutura CLI implementada, mas requer algoritmo de detecção de tabelas

- **Sistema de Logging JSON:** Logs detalhados para todas operações

- ■ **Backup Automático:** Backup antes de operações destrutivas

- ■ **Validações Robustas:** Tratamento completo de erros

Melhorado

- ■ **Extração de objetos:** text, image, link, annotation implementados

- ■ **Edição de texto:** Suporta fonte, cor, tamanho, posição, rotação, alinhamento, padding

- ■ **Substituição de imagem:** Suporta filtros grayscale e invert

Documentado

- ■■ `edit-table`: Limitação técnica conhecida (requer algoritmo de detecção de tabelas)

[0.2.0] - 2025-01-XX (Fase 2 - Modelos e Schemas)

Adicionado

- ■ **Modelos de Dados Completos:** TextObject, ImageObject, TableObject, LinkObject, FormFieldObject, GraphicObject, LayerObject, FilterObject, AnnotationObject

- ■ **Classes de Exceções Customizadas:** TextNotFoundError, PaddingError, InvalidPageError, etc.

- ■ **Serialização JSON:** Métodos `to_dict()` e `from_dict()` para todos os modelos

- ■ **Banner ASCII:** Banner personalizado exibido no CLI

Melhorado

- ■ **Estrutura de dados:** Uso de dataclasses com type hints

- ■ **Validação:** Exceções específicas para diferentes cenários de erro

[0.1.0] - 2025-01-XX (Fase 1 - Estrutura Inicial)

Adicionado

- ■ Estrutura básica do projeto (`src/`, `tests/`, `examples/`)
 - ■ CLI básico com Typer
 - ■ Arquitetura modular (`core`, `app`, `CLI`)
 - ■ Dependências: PyMuPDF, PyPDF2, Typer, Rich
 - ■ README inicial
 - ■ `.cursorrules` com padrões de desenvolvimento
-

Formato

O formato é baseado em [Keep a Changelog](#), e este projeto adere ao [Semantic Versioning](#).

Tipos de Mudanças

- ■ **Adicionado**: Para novas funcionalidades
- ■ **Melhorado**: Para mudanças em funcionalidades existentes
- ■■ **Documentado**: Para limitações técnicas conhecidas
- ■ **Corrigido**: Para correção de bugs
- ■■ **Removido**: Para funcionalidades removidas
- ■ **Segurança**: Para vulnerabilidades corrigidas